Continuous Integration (CI)

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

NORTHEASTERN.

Learning Objectives

- Define Continuous Integration and explain its core principles
- Describe the key practices and benefits of CI in cloud-native development
- Understand the CI pipeline workflow and its components
- Identify common CI tools and their use cases
- Connect CI practices to the DevOps and Twelve-Factor App methodologies covered previously

What is Continuous Integration?

A software development practice where developers frequently integrate their code changes into a shared repository, typically multiple times per day. Each integration is verified by an automated build and automated tests.

The Three Pillars of CI

1. **Frequent Commits**
   - Integrate code at least daily, ideally multiple times per day
   - Small, incremental changes rather than large batches
   - Connection to Git workflow covered earlier
2. **Automated Build**
   - Every commit triggers an automated build process
   - Build must be reproducible and consistent
   - Connection to Twelve-Factor App (Factor III: Config, Factor V: Build/Release/Run)
3. **Automated Testing**
   - Tests run automatically on every integration
   - Fast feedback on code quality
   - Tests must pass before code is considered "integrated"

Core CI Practices

Maintain a Single Source Repository

- All code in version control (Git)
- Single source of truth for the project
- **Anti-pattern:** Multiple disconnected repositories for components that need to work together

Automate the Build

- One command should build the entire system
- Build scripts live in the repository

Example workflow:

```
git push → trigger build → compile code → package artifact
```

Make the Build Self-Testing

- Automated tests run as part of the build
- Types of tests in CI:
  - **Unit tests:** Fast, isolated tests of individual components
  - **Integration tests:** Verify components work together
  - **Linting/Static analysis:** Code quality checks
- **Key principle:** A build that passes tests but has no tests is not truly validated

Everyone Commits Frequently

- Frequent integration reduces merge complexity
- Shared code ownership
- **Psychological benefit:** Less fear of integration

Keep the Build Fast

- Target: Under 10 minutes for primary feedback
- Fast feedback = faster development cycles
- **Trade-off:** Comprehensive testing vs. build speed
- **Solution:** Parallel test execution, test prioritization

CI Pipeline Components

The CI Pipeline Flow

Trigger

- **Push to repository:** Most common trigger
- **Pull request creation:** For code review workflows
- **Scheduled:** Nightly builds, periodic full test suites
- **Manual:** On-demand builds for specific scenarios

Build

- **Checkout code** from repository
- **Install dependencies** (npm install, pip install, etc.)
- **Compile/transpile** if needed
- **Create executable artifact** (JAR, binary, container image)

Test

- **Unit tests:** Fastest, run first
- **Integration tests:** Verify component interactions
- **Code coverage:** Measure test completeness
- **Fail fast:** Stop pipeline on first failure (configurable)

Analyze

- **Static code analysis:** Security vulnerabilities, code smells
- **Dependency scanning:** Check for vulnerable libraries
- **Linting:** Enforce code style standards
- **Tools:** SonarQube, Snyk, ESLint, Pylint

Artifact Generation

- **Build artifacts:** Compiled code, packages
- **Container images:** Docker images tagged with commit SHA
- **Artifact storage:** Push to registries (Docker Hub, ECR, GCR)
- **Connection to CD:** Artifacts become input for deployment pipeline

Pipeline as Code

- Define pipeline in version-controlled files
- **Benefit:** Pipeline changes go through same review process as application code

Summary

Key Takeaways

- **CI is about risk reduction** - Small, frequent integrations are less risky than large, infrequent ones
- **Automation is essential** - Manual builds and tests don't scale
- **Fast feedback enables agility** - Developers need to know quickly if their changes broke something
- **CI is foundational for CD** - You can't continuously deploy if you can't continuously integrate
- **Pipeline as code** - Treat your CI configuration like application code

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS